



400-650-7088

***Tong*Search** V3.0

脚本使用手册

版本：3061A01

2025年09月

声明

版权

Copyright ©2025北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech** 标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

手册版本	适用产品	更新内容
3061A01	3.0.6.1	修改版权信息
3060A01	3.0.6.0	无
3050A01	3.0.5.0	无
3040A01	3.0.4.0	无
3030A01	3.0.3.0	无
3020A01	3.0.2.0	无
3010A01	3.0.1.0	首次发版。

前言

本文档是 TongSearch 产品的用户手册之一，提供命令行工具的使用说明，所有命令行工具的功能说明、相关参数、实例及验证方法。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

- 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

- 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

- 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

- 基本系统管理任务
- 掌握 Linux 系统基本操作
- 在终端窗口发布命令
- 使用 Web 浏览器
- 安装软件

- 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

- 实践经验

为了最大程度地受益于本文档，建议您具备一定实践经验。这将帮助您更好地应用文档中的操作指南 and 建议。

请注意，本文档不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容 or 所需背景有任何疑问，请在使用之前咨询相关专业人士 or 寻求额外的支持。

用户手册集

TongSearch V3.0 为您提供的用户使用手册集包含的文档有：

编号	手册集文档	说明
1	000_TongSearch_V3.0产品白皮书	介绍了 TongSearch 的产品概述、产品功能、产品特色及产品小结。

编号	手册集文档	说明
2	001_TongSearch_V3.0产品介绍手册	介绍了 TongSearch 概述、功能、场景、基本概念等。
3	002_TongSearch_V3.0快速使用手册	介绍了如何快速上手 TongSearch。
4	003_TongSearch_V3.0产品安装手册	提供 TongSearch 常见场景的部署方式及示例程序使用说明。
5	004_TongSearch_V3.0集群管理手册	提供 TongSearch 集群搭建、灾备、升级说明。
6	005_TongSearch_V3.0DSL使用手册	提供 TongSearch 产品的 DSL 语句使用说明。
7	006_TongSearch_V3.0EQL使用手册	提供 TongSearch 产品的 EQL 语句使用说明。
8	007_TongSearch_V3.0SQL使用手册	提供 TongSearch 产品的 SQL 语句使用说明。
9	008_TongSearch_V3.0脚本使用手册	提供命令行工具的使用说明，所有命令行工具的功能说明、相关参数、实例及验证方法。
10	009_TongSearch_V3.0聚合使用手册	介绍了如何使用聚合语句操作 TongSearch 中存储的数据。
11	010_TongSearch_V3.0REST API 手册	提供了通过 HTTP 公开的 REST API。
12	011_TongSearch_V3.0Java SDK 手册	提供基于 Java 语言的客户端使您能够轻松的访问 TongSearch 的 API。
13	012_TongSearch_V3.0工具使用手册	介绍了 TongSearch 提供的各种脚本工具使用说明。
14	013_TongSearch_V3.0数据迁移手册	提供 Elasticsearch 中存储的数据向 TongSearch 迁移的方案。
15	014_TongSearch_V3.0安全管理手册	提供 TongSearch 安全特性使您能够轻松保护集群。
16	015_TongSearch_V3.0性能优化手册	介绍了在使用 TongSearch 过程中的写优化、读优化、服务器配置优化、数据结构优化以及集群架构优化建议。
17	016_TongSearch_V3.0监控告警手册	提供了 Prometheus、Alertmanager 以及 Grafana 结合使用对 TongSearch 进行监控的说明。
18	017_TongSearch_V3.0常见问题手册	提供使用 TongSearch 过程中遇到的常见问题及解决方案。

编号	手册集文档	说明
19	018_TongSearch_V3.0控制台使用手册	提供管理和监视 TongSearch 集群的用户界面，旨在简化 TongSearch 的配置、维护和监控。
20	019_TongSearch_V3.0TongPipeline使用手册	提供 TongPipeline 详细的使用说明，帮助用户快速上手 TongPipeline 并深入了解其各项功能。
21	020_TongSearch_V3.0插件使用手册	提供 TongSearch 插件详细的使用说明。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

- 网址：www.tongtech.com
- 电话：400-650-7088
- 邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

- 您的姓名
- 您的公司信息
- 您的联系方式
- 操作系统及其版本
- 产品版本号
- 日志等错误的详细信息

目录

声明	1
版本变更说明	2
前言	3
1. 如何使用脚本	8
1.1. 脚本介绍	8
1.2. 脚本使用方式	8
1.3. 脚本参数	9
1.4. 短脚本形式	10
1.5. 存储的脚本	11
1.6. 脚本用法	11
1.7. 搜索模板	12
1.8. 脚本缓存	12
1.9. 脚本错误	12
2. 访问文档字段和特殊变量	14
2.1. 更新脚本	14
2.2. 搜索和聚合脚本	14
2.3. 脚本内获取文档得分	14
2.4. 文档值	15
2.5. 文档的源	16
2.6. 存储的字段	17
3. 脚本与安全	19
3.1. 经验法则	19
3.2. 其它安全层	19
3.3. 允许脚本类型设置	20
3.4. 允许脚本上下文设置	20
4. painless 脚本安全	21
5. Lucene 表达式语言	22
5.1. 性能	22
5.2. 语法	22
5.3. numeric 字段 API	22
5.4. date 字段 API	23
5.5. geo_point 字段 API	24
6. 高级脚本使用脚本引擎	25
6.1. 高级脚本使用脚本引擎用法	25
6.1.1. 脚本编写	25
6.1.2. 部署到 TS	28

6.1.3. 执行脚本 28

1. 如何使用脚本

1.1. 脚本介绍

通过使用脚本，用户可以在 TongSearch 中计算自定义表达式。例如，您可以使用脚本将计算结果作为字段返回，或者评估查询的自定义分数。

默认的脚本语言是 Painless。附加的 lang 插件使您能够运行用其他语言编写的脚本。在任何可以使用脚本的地方，都可以包含一个 lang 参数来指定脚本的语言。

通用语言

这些语言可以在脚本 API 中用于任何目的，并提供最大的灵活性。

语言	是否沙箱化	必要插件
painless	是	(内置，不需要)

特殊用途的语言

这些语言不太灵活，但对于某些任务来说通常具有更好的性能。

语言	是否沙箱化	必要插件	目的
expression	是	(内置)	快速自定义排名和排序
mustache	是	(内置)	模板
java	否	自己编写	专家的 API

 沙箱化语言的设计考虑了安全性。非沙箱化语言可能是一个安全问题，请阅读脚本和安全性以了解更多细节。

1.2. 脚本使用方式

只要是 TongSearch API 中支持脚本的地方，语法都遵循相同的模式：

```
"script": {
  "lang": "...",    ①
  "source" | "id": "...", ②
  "params": { ... }  ③
}
```

- ① 编写脚本的语言，默认为“painless”。
- ② 可以被指定为内联脚本的 source 或存储脚本的 id 来表示脚本本身。
- ③ 应该传递到脚本中的任何命名参数。

例如，下面的脚本在搜索请求中用于返回脚本字段（scripted field）：

```
PUT my_index/_doc/1
{
  "my_field": 5
}

GET my_index/_search
{
  "script_fields": {
    "my_doubled_field": {
      "script": {
        "lang": "expression",
        "source": "doc['my_field'] * multiplier",
        "params": {
          "multiplier": 2
        }
      }
    }
  }
}
```

1.3. 脚本参数

参数	说明
lang	指定编写脚本的语言。默认为“painless”。
source, id	指定脚本的源（source）。 如上例所示，内联（inline）脚本被指定为源（source）。存储的（stored）脚本是指定 id，并从集群中检索。
params	指定作为变量传递到脚本中的任何命名参数。

注意事项

首选参数：TongSearch 第一次看到一个新脚本时，它会编译它并将编译后的版本存储在缓存中。编译可能是一个繁重的处理过程。

如果您需要将变量传递到脚本中，应该将它们作为命名的 params 传递，而不是将值硬编码到脚本本身中。例如，如果您希望能够将字段值乘以不同的乘数，请不要将乘数硬编码到脚本中：

```
"source": "doc['my_field'] * 2"
```

相反，将其作为命名参数传入：

```
"source": "doc['my_field'] * multiplier",
"params": {
```

```
"multiplier": 2
}
```

每当乘数改变时，第一个版本必须重新编译，而第二个版本只编译一次，可以减少编译的时间。

如果您在很短的时间内编译了太多不同的脚本，TongSearch 将拒绝新的动态脚本，并抛出一个 `circuit_breaking_exception` 错误。

默认情况下，每分钟最多可以编译 15 个内联脚本。可以通过设置 `script.max_compilations_rate` 来动态更改此设置。

1.4. 短脚本形式

使用 Painless 原生的语法功能，您可以减少脚本中的冗长内容并使其更短。

下面是一个简化的脚本：

```
GET my-index-000001/_search{
  "script_fields": {
    "my_doubled_field": {
      "script": {
        "lang": "painless",
        "source": "return doc['my_field'].value * params.get('multiplier');",
        "params": {
          "multiplier": 2
        }
      }
    }
  }
}
```

而相同脚本的一般形式是：

```
GET my-index-000001/_search{
  "script_fields": {
    "my_doubled_field": {
      "script": {
        "source": "doc['my_field'].value * params['multiplier']",
        "params": {
          "multiplier": 2
        }
      }
    }
  }
}
```

这个版本的脚本删除了几个组件，并显著简化了语法：

- lang: 因为 Painless 是默认语言, 所以如果要编写一个 Painless 脚本, 就不需要指定语言。
- return 关键字: Painless 自动使用脚本中最后的一个语句 (如果可能的话), 在需要返回值的脚本上下文中生成返回值。
- get 方法: 用[]代替, Painless 为 Map 类型提供了一个快捷方式, 允许使用方括号来代替冗长的 get 方法。
- 源语句末尾的分号: painless 不需要在语句块的最后使用分号。但是在其他情况下, 确实需要它们来消除歧义。

1.5. 存储的脚本

您可以使用存储的脚本 API 从集群状态中存储和检索脚本。

若启用了 TongSearch 安全功能, 用户必须拥有以下权限才能创建、检索和删除存储的脚本:

cluster: all 或 manage

1.6. 脚本用法

下面是一个通过 “/_scripts/{id}” 使用存储脚本的示例。

首先, 在集群状态中创建名为 “calculate-score” 的脚本:

```
POST _scripts/calculate-score
{
  "script": {
    "lang": "painless",
    "source": "Math.log(_score * 2) + params.my_modifier"
  }
}
```

用户还可以指定一个上下文作为 url 路径的一部分, 以 “/_scripts/{id}/{context}” 的形式。根据该特定上下文编译存储的脚本。

注意:

上下文不能随便写。

```
POST _scripts/calculate-score/score
{
  "script": {
    "lang": "painless",
    "source": "Math.log(_score * 4) + params.my_modifier"
  }
}
```

同样的脚本可以通过以下方式检索:

```
GET _scripts/calculate-score
```

可以通过指定参数 id 来使用存储的脚本，如下所示。

```
GET twitter/_search
{
  "query": {
    "script_score": {
      "query": {
        "match": {
          "message": "some message"
        }
      },
      "script": {
        "id": "calculate-score",
        "params": {
          "my_modifier": 2
        }
      }
    }
  }
}
```

删除脚本：

```
DELETE _scripts/calculate-score
```

1.7. 搜索模板

用户可以使用“_scripts API”来存储搜索模板(search templates)。

用户可以使用存储的搜索模板来运行搜索，而无需写出整个查询。只需提供存储模板的 ID 和模板参数。当用户希望快速无误地运行常用查询时，这很有用。

1.8. 脚本缓存

默认情况下，所有脚本都会被缓存，它们只需要在变更时重新编译。

默认情况下，脚本没有过期时间，但是你可以通过使用 script.cache.expire 设置来更改此行为。

可以使用 script.cache.max_size 设置来配置此缓存的个数。默认情况下，缓存的脚本的个数为100。



脚本的大小限制为65,535字节。这可以通过设置script.max_size_in_bytes设置来增加该软限制，但如果脚本非常大，则应考虑使用原生脚本引擎(native script engine)。

1.9. 脚本错误

当存在编译或运行时异常时，TongSearch 会返回错误的详细信息。此响应的内容有助于跟踪问题。

2. 访问文档字段和特殊变量

根据脚本的使用场合，它可以访问某些特殊变量和文档字段。

2.1. 更新脚本

在更新（update）、按查询更新（update-by-query）或重新索引（reindex）中使用的脚本可以访问变量 ctx，该变量公开了以下属性。

属性	属性说明
ctx._source	访问 _source 字段。
ctx.op	应该应用于文档的操作：index 或 delete。
ctx._index 等	访问文档元字段，其中一些可能是只读的。

2.2. 搜索和聚合脚本

除了每次搜索命中执行一次的脚本字段，搜索和聚合中使用的脚本将对每个匹配查询或聚合的文档都执行一次。这取决于您有多少文档，这可能意味着数百万或数十亿次执行：这些脚本需要运行得很快！

可以使用文档值（doc-values）、源（_source）字段或存储的字段（stored fields）从脚本中访问字段值，下面会对他们进行一一说明。

2.3. 脚本内获取文档得分

function_score 查询、基于脚本的排序或聚合中使用的脚本，可以访问代表文档当前相关性分数的 _score 变量。

下面是一个在 function_score 查询中使用脚本来改变每个文档的相关性评分（_score）的示例。

```
PUT my_index/_doc/1?refresh
{
  "text": "quick brown fox",
  "popularity": 1
}

PUT my_index/_doc/2?refresh
{
  "text": "quick fox",
  "popularity": 5
}

GET my_index/_search
```

```
{
  "query": {
    "function_score": {
      "query": {
        "match": {
          "text": "quick brown fox"
        }
      },
      "script_score": {
        "script": {
          "lang": "expression",
          "source": "_score * doc['popularity']"
        }
      }
    }
  }
}
```

2.4. 文档值

到目前为止，从脚本中访问字段值最快最有效的方法是使用 `doc['field_name']` 语法，该语法从文档值（doc values）中检索字段值。

文档值是按列式字段值存储的，默认情况下，除了 analyzed text 字段（会被分析的字段）之外，所有字段都启用该存储方式。

```
PUT my_index/_doc/1?refresh
{
  "cost_price": 100
}

GET my_index/_search
{
  "script_fields": {
    "sales_price": {
      "script": {
        "lang": "expression",
        "source": "doc['cost_price'] * markup",
        "params": {
          "markup": 0.2
        }
      }
    }
  }
}
```


文档值只能返回“简单”的字段值，如：numbers、dates、geo-points、terms 等。

若字段是多值的，则返回这些值的数组。它不能返回 JSON 对象。

注意事项

- 缺失的字段

如果映射中缺少 field，doc['field'] 将会引发错误。在 painless 下，可以首先使用 doc.containsKey('field') 进行检查，以防止访问 doc 的字段不存在。但仍然没有办法在 expression 脚本中检查映射中是否存在该字段。

- 文档值和 text 字段

若启用了 fielddata，则 doc['field'] 语法也可以用于分析的 (analyzed)text 字段。

注意：

在 text 字段上启用 fielddata 需要将所有的词项加载到 JVM 堆中，这在内存和 CPU 方面的开销都很大，并且在脚本中访问 text 字段没有意义。

2.5. 文档的源

可以使用 _source.field_name 语法访问文档的源（_source）。

_source 作为映射的映射加载，因此对象字段中的属性可以像 _source.name.first 这样来访问。

首选文档值(doc-values)而非 _source

访问 _source 字段比使用文档值（doc-values）要慢得多。

_source 字段针对每个结果返回几个字段进行了优化，而文档值（doc values）针对访问许多文档中特定字段的值进行了优化。

当从搜索结果中为前10个命中生成脚本字段(script field)时，使用 _source 是有意义的，但是对于其他搜索和聚合用例，总是首选使用文档值(doc values)。

示例说明

```
PUT my_index
{
  "mappings": {
    "properties": {
      "first_name": {
        "type": "text"
      },
      "last_name": {
        "type": "text"
      }
    }
  }
}
```

```

}

PUT my_index/_doc/1?refresh
{
  "first_name": "Barry",
  "last_name": "White"
}

GET my_index/_search
{
  "script_fields": {
    "full_name": {
      "script": {
        "lang": "painless",
        "source": "params._source.first_name + ' ' + params._source.last_name"
      }
    }
  }
}

```

2.6. 存储的字段

存储字段——在映射中明确标记为 ("store": true) 的字段，可以使用 `_fields['field_name'].value` 或 `_fields['field_name']` 语法进行访问。

```

PUT my_index
{
  "mappings": {
    "properties": {
      "full_name": {
        "type": "text",
        "store": true
      },
      "title": {
        "type": "text",
        "store": true
      }
    }
  }
}

PUT my_index/_doc/1?refresh
{
  "full_name": "Alice Ball",

```

```
"title": "Professor"
}

GET my_index/_search
{
  "script_fields": {
    "name_with_title": {
      "script": {
        "lang": "painless",
        "source": "params._fields['title'].value + ' ' + params._fields['full_name'].value"
      }
    }
  }
}
```

存储的(stored)与源(_source)

_source 字段只是一个特殊的存储的字段，因此性能和其他存储字段相似。_source 提供了对被索引的原始文档主体的访问（包括区分 null 值与空字段、单值数组与普通标量等的功能）。

只有当 _source 非常大，并且访问一些小的存储字段比访问整个 _source 成本更低时，使用存储字段而不是 _source 字段才有意义。

3. 脚本与安全

虽然 TongSearch 尽一切努力防止脚本失控，但安全最好是分层完成的事情，因为所有软件都有漏洞，最大限度地降低任何安全层中的失败风险是很重要的。

下面是一些避免 TongSearch 成为漏洞的经验法则。

3.1. 经验法则

1. 不要以 root 身份运行

首先，也最重要的是，永远不要以 root 用户的身份运行 TongSearch，因为这将允许绕过其他安全层，并尝试在服务器上执行任何操作。

如果 TongSearch 检测到它以 root 用户身份运行，就会拒绝启动，但这非常重要。

2. 不要直接向用户公开 TongSearch

不要直接向用户公开 TongSearch，而是让一个应用程序代表用户发出请求。

如果这是不可能的，请使用一个应用程序来净化来自用户的请求。

如果这是不可能的，那么就需要一些机制来跟踪哪些用户做了什么。编写一个 `_search` 来压垮 TongSearch 并关闭集群是有可能的。

所有这样的搜索都应该被认为是缺陷，TongSearch 努力阻止这种情况，但它们仍然是可能的。

3. 不要将 TongSearch 直接暴露在互联网上

不要将 TongSearch 暴露给互联网，而是让一个应用程序代表互联网发出请求。

不要考虑让应用程序“净化”对 TongSearch 的请求。

一个非常有决心的恶意用户可能会编写搜索，使 TongSearch 集群不堪重负，并使其崩溃。例如：

好的做法：

- 用户在搜索框中键入文本，文本将被直接发送到匹配(match)、匹配短语(match phrase)、简单查询字符串查询或任何的 suggesters 查询。
- 运行包含上述任何查询的脚本，该脚本是作为应用程序开发过程的一部分编写的。
- 使用用户提供的 params 运行脚本。
- 用户操作使文档具有固定的结构。

坏的做法：

- 用户可以编写任意脚本、查询和 `_search` 请求。
- 用户操作使文档具有用户自定义的结构。

3.2. 其它安全层

除了用户权限和脚本沙箱，TongSearch 使用 Java 安全管理器和本地安全工具作为额外的安全层。

作为启动序列的一部分，TongSearch 启用了 Java 安全管理器，它限制了部分代码可以采取的动作。painless 使用这个来限制生成的 painless 脚本可以采取的动作，防止它们能够做像写文件和监听套接字这样的事情。

TongSearch 在 Linux 中使用 seccomp、在 macOS 中使用 Seatbelt、在 Windows 中使用 ActiveProcessLimit 来防止 TongSearch fork 或执行其他进程。

下面将会描述脚本的安全设置，以及如何改变上述的默认设置。当允许超过默认值时，您应该非常非常小心。任何额外的权限都会削弱 TongSearch 部署的整体安全性。

3.3. 允许脚本类型设置

默认情况下允许执行所有脚本类型。这可以使用设置 `script.allowed_types` 来修改。

只允许执行设置中指定的类型。

若要指定不允许任何类型，请将 `script.allowed_types` 设置为 `none`。

```
script.allowed_types: inline
```

这将只允许执行内联脚本，而不允许执行存储的脚本（及任何其他类型的脚本）。

3.4. 允许脚本上下文设置

默认情况下，允许执行所有脚本上下文。这可以使用设置 `script.allowed_contexts` 来修改。只允许执行设置中指定的上下文。要指定不允许任何上下文，请将 `script.allowed_contexts` 设置为 `none`。

```
script.allowed_contexts: score, update
```

这将只允许执行评分（score）和更新（update）脚本，而不允许执行聚合及插件脚本（以及任何其他上下文）。

4. painless 脚本安全

painless 是一种简单、安全的脚本语言，专门设计用于 TongSearch。它是 TongSearch 的默认脚本语言，可以安全地用于内联和存储的脚本。

用户可以在任何可以在 TongSearch 中使用脚本的地方使用 painless。

painless 提供了：

1. 快速性能：painless 脚本的运行速度比替代方案快几倍。
2. 安全：具有方法调用/字段粒度的细粒度 允许列表（allowlists）。有关可用类和方法的完整列表。
3. 可选类型：变量和参数可以使用显式类型或动态(定义)的类型。
4. 语法：扩展 Java 语法的子集，以提供额外的脚本语言特性。
5. 优化：为 TongSearch 脚本设计。

5. Lucene 表达式语言

Lucene 的表达式将 javascript 表达式编译成字节码。

它们是为高性能定制排名（ranking）和排序（sorting）功能而设计的，默认情况下支持内联（inline）和存储的（stored）脚本。

5.1. 性能

表达式被设计成具有与自定义 Lucene 代码相媲美的性能。这种性能是因为与其他脚本引擎相比，每个文档的开销较低：表达式做了更多的“预先准备”。

这使得执行速度非常快，甚至比编写原生（native）脚本还要快。

5.2. 语法

表达式支持 javascript 语法的子集：单个表达式。

expression 脚本中的变量可用于访问：

- 文档的字段，比如：doc['myfield'].value
- 字段支持的变量和函数，比如：doc['myfield'].empty
- 传递给脚本的参数，比如：mymodifier
- 当前文档的得分 _score（仅在 script_score 中使用时有效）

可以将表达式脚本用于 script_score、script_fields、排序脚本和数值聚合脚本，只需将参数 lang 设置为 expression。

5.3. numeric 字段 API

表达式	描述
doc['field_name'].value	字段的值，双精度（double）值。
doc['field_name'].empty	一个布尔值，指示该字段在文档中是否为空值。
doc['field_name'].length	该文档中值的数量。
doc['field_name'].min()	该文档中字段的最小值。
doc['field_name'].max()	该文档中字段的最大值。
doc['field_name'].median()	该文档中字段值的中位数。
doc['field_name'].avg()	该文档中字段值的平均数。
doc['field_name'].sum()	该文档中字段值的总和。

当文档完全缺少该字段时，默认情况下，该值将被视为 0。您也可以将其视为另一个值。

例如：doc['myfield'].empty ? 100 : doc['myfield'].value

当文档的字段有多个值时，默认情况下返回最小值。您也可以选择一个不同的值。

例如：doc['myfield'].sum()。

布尔(bool)字段显示为数字，true 映射为 1，false 映射为 0。

例如：doc['on_sale'].value ? doc['price'].value * 0.5 : doc['price'].value。

5.4. date 字段 API

date 字段被视为自 1970 年 1 月 1 日以来的毫秒数，并支持上述 numeric 字段 API 以及对某些特定的日期字段的访问。

表达式	描述
doc['field_name'].date.centuryOfEra	世纪 (1-2920000)。
doc['field_name'].date.dayOfMonth	天 (1-31)，比如 1 表示每个月的第一天。
doc['field_name'].date.dayOfWeek	星期几 (1-7)，比如 1 代表星期一。
doc['field_name'].date.dayOfYear	一年中的某一天，比如 1 表示一月1号。
doc['field_name'].date.era	纪元，0 表示公元前(BC)，1 表示公元后(AD)。
doc['field_name'].date.hourOfDay	小时(0-23)。
doc['field_name'].date.millisOfDay	一天中的毫秒数(0-86399999)。
doc['field_name'].date.millisOfSecond	一秒内的毫秒(0-999)。
doc['field_name'].date.minuteOfDay	一天中的分钟(0-1439)。
doc['field_name'].date.minuteOfHour	一小时内的分钟(0-59)。
doc['field_name'].date.monthOfYear	一年中的月数(1-12)，比如 1 表示一月份。
doc['field_name'].date.secondOfDay	一天中的秒数(0-86399)。
doc['field_name'].date.secondOfMinute	一分钟内的秒数(0-59)。
doc['field_name'].date.year	年(-292000000 - 292000000)。
doc['field_name'].date.yearOfCentury	一个世纪中的年数(1-100)。
doc['field_name'].date.yearOfEra	一个纪元中的年(1-292000000)。

下面这个例子显示了date字段 date0 和 date1 之间的年份差异：

`doc['date1'].date.year - doc['date0'].date.year`

5.5. geo_point 字段 API

表达式	描述
<code>doc['field_name'].empty</code>	一个布尔值，指示该字段在文档中是否为空值。
<code>doc['field_name'].lat</code>	地理位置坐标点的纬度值。
<code>doc['field_name'].lon</code>	地理位置坐标点的经度值。

下面这个例子计算与华盛顿市区的距离（单位为公里）：

`haversin(38.9072, 77.0369, doc['field_name'].lat, doc['field_name'].lon)`

在这个例子中，坐标可以作为参数传递给脚本，例如：基于用户的地理位置。

局限性

相对于其他脚本语言有一些限制：

只能访问 `numeric`、`boolean`、`date` 及 `geo_point` 字段。

存储的字段不可用。

6. 高级脚本使用脚本引擎

ScriptEngine 是实现脚本语言的后端。它还可以用来编写需要使用高级脚本内部机制的脚本。例如，一个脚本希望在评分时使用词项的频率。

插件文档中有更多关于如何编写插件的信息，这样 TongSearch 就可以正确地加载它。

要注册 ScriptEngine，插件应该实现 ScriptPlugin 接口并重写 getScriptEngine(Settings settings) 方法。

6.1. 高级脚本使用脚本引擎用法

下面是一个使用语言名称为 expert_scripts 的自定义脚本引擎的示例。它实现了一个名为 pure_df 的脚本，该脚本可以用作搜索脚本来覆盖每个文档的分数，作为所提供词项的文档频率。

6.1.1. 脚本编写

```
private static class MyExpertScriptEngine implements ScriptEngine {
    @Override
    public String getType() {
        return "expert_scripts";
    }

    @Override
    public <T> T compile(
        String scriptName,
        String scriptSource,
        ScriptContext<T> context,
        Map<String, String> params
    ) {
        if (context.equals(ScoreScript.CONTEXT) == false) {
            throw new IllegalArgumentException(getType()
                + " scripts cannot be used for context ["
                + context.name + "]);
        }
        // we use the script "source" as the script identifier
        if ("pure_df".equals(scriptSource)) {
            ScoreScript.Factory factory = new PureDfFactory();
            return context.factoryClazz.cast(factory);
        }
        throw new IllegalArgumentException("Unknown script name "
            + scriptSource);
    }

    @Override
```

```

public void close() {
    // optionally close resources
}

@Override
public Set<ScriptContext<?>> getSupportedContexts() {
    return Collections.singleton(ScoreScript.CONTEXT);
}

private static class PureDfFactory implements ScoreScript.Factory,
                                           ScriptFactory {

    @Override
    public boolean isResultDeterministic() {
        // PureDfLeafFactory only uses deterministic APIs, this
        // implies the results are cacheable.
        return true;
    }

    @Override
    public LeafFactory newFactory(
        Map<String, Object> params,
        SearchLookup lookup
    ) {
        return new PureDfLeafFactory(params, lookup);
    }
}

private static class PureDfLeafFactory implements LeafFactory {
    private final Map<String, Object> params;
    private final SearchLookup lookup;
    private final String field;
    private final String term;

    private PureDfLeafFactory(
        Map<String, Object> params, SearchLookup lookup) {
        if (params.containsKey("field") == false) {
            throw new IllegalArgumentException(
                "Missing parameter [field]");
        }
        if (params.containsKey("term") == false) {
            throw new IllegalArgumentException(
                "Missing parameter [term]");
        }
        this.params = params;
        this.lookup = lookup;
    }
}

```

```

        field = params.get("field").toString();
        term = params.get("term").toString();
    }

    @Override
    public boolean needs_score() {
        return false; // Return true if the script needs the score
    }

    @Override
    public ScoreScript newInstance(LeafReaderContext context)
        throws IOException {
        PostingsEnum postings = context.reader().postings(
            new Term(field, term));
        if (postings == null) {
            /*
             * the field and/or term don't exist in this segment,
             * so always return 0
             */
            return new ScoreScript(params, lookup, context) {
                @Override
                public double execute(
                    ExplanationHolder explanation
                ) {
                    return 0.0d;
                }
            };
        }
        return new ScoreScript(params, lookup, context) {
            int currentDocid = -1;
            @Override
            public void setDocument(int docid) {
                /*
                 * advance has undefined behavior calling with
                 * a docid <= its current docid
                 */
                if (postings.docID() < docid) {
                    try {
                        postings.advance(docid);
                    } catch (IOException e) {
                        throw new UncheckedIOException(e);
                    }
                }
                currentDocid = docid;
            }
        }
    }

```

```
@Override
public double execute(ExplanationHolder explanation) {
    if (postings.docID() != currentDocid) {
        /*
         * advance moved past the current doc, so this
         * doc has no occurrences of the term
         */
        return 0.0d;
    }
    try {
        return postings.freq();
    } catch (IOException e) {
        throw new UncheckedIOException(e);
    }
}
};
}
```

6.1.2. 部署到 TS

1. 把脚本打包为 jar 包。
2. 把包复制到 modules 目录下，并新建配置文件 plugin-descriptor.properties，并且在配置文件中，指定 classname 为 jar 包中的 MyExpertScriptEngine 全限定类名。

6.1.3. 执行脚本

可以通过将脚本的 lang 指定为 expert_scripts，并将脚本的名称指定为脚本源来执行脚本：

```
POST /_search
{
  "query": {
    "function_score": {
      "query": {
        "match": {
          "body": "foo"
        }
      },
      "functions": [
        {
          "script_score": {
            "script": {
              "source": "pure_df",
              "lang": "expert_scripts",
            }
          }
        }
      ]
    }
  }
}
```

```
    "params": {  
      "field": "body",  
      "term": "foo"  
    }  
  }  
}  
]  
}  
}
```

